

Software Testing Techniques

Software Testing Techniques: Black-box Testing, White box testing, Gray box Testing, Boundary Value Analysis, Regression Testing, Cause Effect graphing, CRUD, Database, Mutation Testing, Random Testing, Performance testing, Stress Testing, Spike Testing, Accepting Testing, Alpha and Beta Testing, V& V

BLACK Box Testing

Black Box Testing is a software testing method in which the functionalities of software applications are tested without having knowledge of internal code structure, implementation details and internal paths. Black Box Testing mainly focuses on input and output of software applications and it is entirely based on software requirements and specifications. It is also known as Behavioral Testing.



The above Black-Box can be any software system you want to test. For Example, an operating system like Windows, a website like Google, a database like Oracle or even your own custom application. Under BlackBox Testing, you can test these applications by just focusing on the inputs and outputs without knowing their internal code implementation. Consider the following video tutorial-

How to do BlackBox Testing

Here are the generic steps followed to carry out any type of Black Box Testing.

- Initially, the requirements and specifications of the system are examined.

- ☐ Tester chooses valid inputs (positive test scenario) to check whether SUT processes them correctly. Also, some invalid inputs (negative test scenario) are chosen to verify that the SUT is able to detect them.
- ☐ Tester determines expected outputs for all those inputs.
- ☐ Software tester constructs test cases with the selected inputs.
- ☐ The test cases are executed.
- ☐ Software tester compares the actual outputs with the expected outputs.
- ☐ Defects if any are fixed and re-tested.

Types of Black Box Testing

There are many types of Black Box Testing but the following are the prominent ones –

- ☐ Functional testing – This black box testing type is related to the functional requirements of a system; it is done by software testers.
- ☐ Non-functional testing – This type of black box testing is not related to testing of specific functionality, but non-functional requirements such as performance, scalability, usability.
- ☐ Regression testing – [Regression Testing](#) is done after code fixes, upgrades or any other system maintenance to check the new code has not affected the existing code.

Tools used for Black Box Testing:

Tools used for Black box testing largely depends on the type of blackbox testing you are doing.

- ☐ For Functional/ Regression Tests you can use – [QTP](#), [Selenium](#)
- ☐ For Non-Functional Tests, you can use – [LoadRunner](#), [Jmeter](#)

Black Box Testing Techniques

Following are the prominent [Test Strategy](#) amongst the many used in Black box Testing

- ☐ Equivalence Class Testing: It is used to minimize the number of possible test cases to an optimum level while maintains reasonable test coverage.
- ☐ Boundary Value Testing: Boundary value testing is focused on the values at boundaries. This technique determines whether a certain range of values are acceptable by the system or not. It is very useful in reducing the number of test cases. It is most suitable for the systems where an input is within certain ranges.
- ☐ Decision Table Testing: A decision table puts causes and their effects in a matrix. There is a unique combination in each column.

White Box Testing

White Box Testing is a testing technique in which software's internal structure, design, and coding are tested to verify input-output flow and improve design, usability, and security. In white box testing, code is visible to testers, so it is also called Clear box testing, Open box testing, Transparent box testing, Code-based testing, and Glass box testing.

It is one of two parts of the Box Testing approach to software testing. Its counterpart, Blackbox testing, involves testing from an external or end-user perspective. On the other hand, White box testing in software engineering is based on the inner workings of an application and revolves around internal testing.

White Box Testing Techniques

A major White box testing technique is Code Coverage analysis. Code Coverage analysis eliminates gaps in a [Test Case](#) suite. It identifies areas of a program that are not exercised by a set of test cases. Once gaps are identified, you create test cases to verify untested parts of the code, thereby increasing the quality of the software product

There are automated tools available to perform [Code coverage analysis](#).

Below are a few coverage analysis techniques a box tester can use:

Statement Coverage:- This technique requires every possible statement in the code to be tested at least once during the testing process of [software engineering](#).

Branch Coverage – This technique checks every possible path (if-else and other conditional loops) of a software application.

Apart from above, there are numerous coverage types such as Condition Coverage, Multiple Condition Coverage, Path Coverage, Function Coverage etc. Each technique has its own merits and attempts to test (cover) all parts of software code. Using Statement and Branch coverage you generally attain 80-90% code coverage which is sufficient.

Following are important WhiteBox Testing Techniques:

- ☐ Statement Coverage
- ☐ Decision Coverage
- ☐ Branch Coverage
- ☐ Condition Coverage
- ☐ Multiple Condition Coverage
- ☐ Finite State Machine Coverage
- ☐ Path Coverage
- ☐ Control flow testing
- ☐ Data flow testing

Types of White Box Testing

White box testing encompasses several testing types used to evaluate the usability of an application, block of code or specific software package. There are listed below —

- ☐ Unit Testing: It is often the first type of testing done on an application. [Unit Testing](#) is performed on each unit or block of code as it is developed. Unit Testing is essentially done by the programmer. As a software developer, you develop a few lines of code, a single function or an object and test it to make sure it works before continuing. Unit Testing helps identify a majority of bugs, early in the software development lifecycle. Bugs identified in this stage are cheaper and easy to fix.
- ☐ Testing for Memory Leaks: Memory leaks are leading causes of slower running applications. A QA specialist who is experienced at detecting memory leaks is essential in cases where you have a slow running software application.

Apart from the above, a few testing types are part of both black box and white box testing. They are listed below

- ☐ White Box [Penetration Testing](#): In this testing, the tester/developer has full information of the application's source code, detailed network information, IP addresses involved and all server information the application runs on. The aim is to attack the code from several angles to expose security threats.
- ☐ White Box Mutation Testing: Mutation testing is often used to discover the best coding techniques to use for expanding a software solution.

White Box Testing Tools

Below is a list of top white box testing tools.

- ☐ [EclEmma](#)
- ☐ [JUnit](#)
- ☐ [PyUnit](#)
- ☐ [HTMLUnit](#)
- ☐ [CppUnit](#)

Advantages of White Box Testing

- ☐ Code optimization by finding hidden errors.
- ☐ White box tests cases can be easily automated.
- ☐ Testing is more thorough as all code paths are usually covered.
- ☐ Testing can start early in SDLC even if GUI is not available.

Disadvantages of White Box Testing

- ☐ White box testing can be quite complex and expensive.

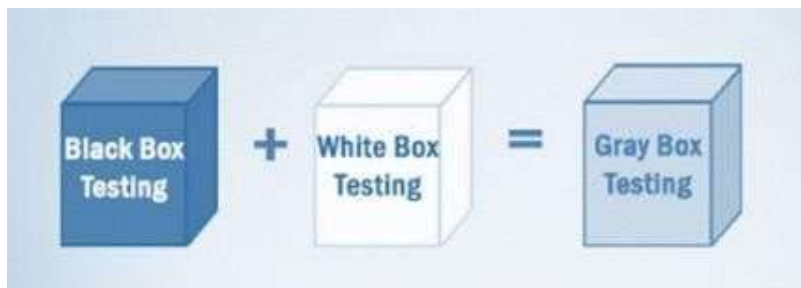
- ❑ Developers who usually execute white box test cases detest it. The white box testing by developers is not detailed and can lead to production errors.
- ❑ White box testing requires professional resources with a detailed understanding of programming and implementation.
- ❑ White-box testing is time-consuming, bigger programming applications take the time to test fully.

Grey Box Testing

Grey Box Testing or Gray box testing is a software testing technique to test a software product or application with partial knowledge of internal structure of the application. The purpose of grey box testing is to search and identify the defects due to improper code structure or improper use of applications. In this process, context-specific errors that are related to web systems are commonly identified. It increases the testing coverage by concentrating on all of the layers of any complex system.

Gray Box Testing is a software testing method, which is a combination of both [White Box Testing](#) and Black Box Testing method.

- ❑ In White Box testing internal structure (code) is known
- ❑ In Black Box testing internal structure (code) is unknown
- ❑ In Grey Box Testing internal structure (code) is partially known



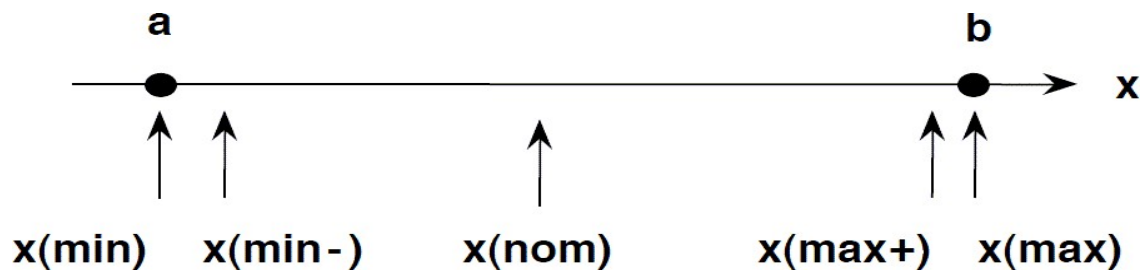
Techniques used for Grey box Testing are-

- ❑ Matrix Testing: This testing technique involves defining all the variables that exist in their programs.
- ❑ Regression Testing: To check whether the change in the previous version has regressed other aspects of the program in the new version. It will be done by testing strategies like retest all, retest risky use cases, retest within a firewall.
- ❑ Orthogonal Array Testing or OAT: It provides maximum code coverage with minimum test cases.
- ❑ Pattern Testing: This testing is performed on the historical data of the previous system defects. Unlike black box testing, gray box testing digs within the code and determines why the failure happened

Boundary Value Analysis Testing

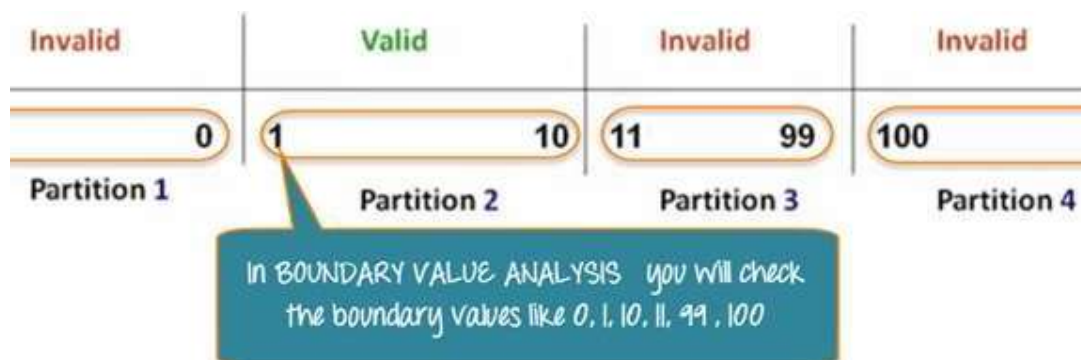
Boundary testing is the process of testing between extreme ends or boundaries between partitions of the input values.

- So these extreme ends like Start- End, Lower- Upper, Maximum- Minimum, Just Inside-Just Outside values are called boundary values and the testing is called “boundary testing”.
- The basic idea in normal boundary value testing is to select input variable values at their:
 1. Minimum
 2. Just above the minimum
 3. A nominal value
 4. Just below the maximum
 5. Maximum



- In Boundary Testing, Equivalence Class Partitioning plays a good role
- Boundary Testing comes after the Equivalence Class Partitioning.

Boundary Value Analysis– in Boundary Value Analysis, you test boundaries between equivalence partitions



In our earlier equivalence partitioning example, instead of checking one value

for each partition, you will check the values at the partitions like 0,1, 10, 11 and so on. As you may observe, you test values at both valid and invalid boundaries. Boundary Value Analysis is also called range checking.

Equivalence partitioning and boundary value analysis(BVA) are closely related and can be used together at all levels of testing.

Example 2: Equivalence and Boundary Value

Following password field accepts minimum 6 characters and maximum 10 characters

That means results for values in partitions 0-5, 6-10, 11-14 should be equivalent

Enter Password:

Submit

Test Scenario #	Test Scenario Description	Expected Outc
1	Enter 0 to 5 characters in password field	System should
2	Enter 6 to 10 characters in password field	System should
3	Enter 11 to 14 character in password field	System should

Examples 3: Input Box should accept the Number 1 to 10

Here we will see the Boundary Value Test Cases

Test Scenario Description	Expected Outcome
Boundary Value = 0	System should NOT accept
Boundary Value = 1	System should accept
Boundary Value = 2	System should accept
Boundary Value = 9	System should accept
Boundary Value = 10	System should accept
Boundary Value = 11	System should NOT accept

Why Equivalence & Boundary Analysis Testing

1. This testing is used to reduce a very large number of test cases to manageable chunks.
2. Very clear guidelines on determining test cases without compromising on the effectiveness of testing.
3. Appropriate for calculation-intensive applications with a large number of variables/inputs.

Regression Testing

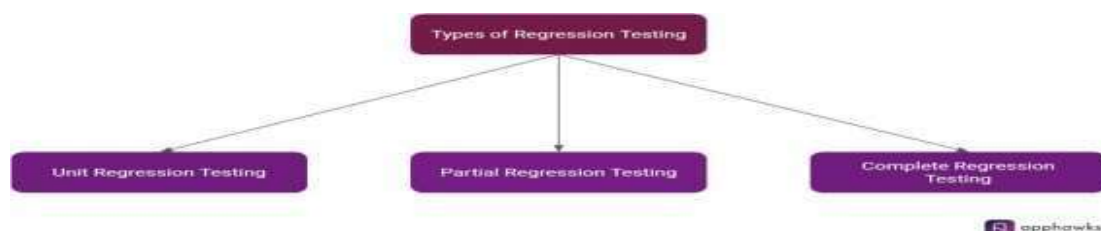
Regression testing is a process in which computer programs are tested in order to ensure that the application or software still works and there are no negative effects on their performance or functionality with the introduction of new changes.

Regression testing is a common part of the process of software development. In larger organizations, regression testing is performed by specialists in code testing. Generally, the applications or software are updated or some new changes are introduced in order to improve its performance so in regression testing the old test cases which are used in testing the previous version of the software are tested in the new version again to ensure that the old performance and capabilities still work.

Regression testing is generally carried out by repeatedly implementing the bits of the code to make sure that the constant modification in code doesn't impact the software's functionality.

Some software developer believes that there is no need for testing everytime there is a new modification in the software until some complications or errors arise in its performance or functionality.

So in simple words, Regression testing is just Re-testing of the software to ensure that any new changes introduced in the software won't affect other functionalities. There are three types of Regression Testing:



The types of regression testing and many more things about regression testing will be discussed in length later in this

types Of Regression Testing

We are already aware of what is regression testing, why we need it, and when it is appropriate to implement. We know that regression testing is performed to check whether the performance of functionality is not affected in a negative way after some new changes are introduced and to identify the bugs due to the changes.

There are three main categories of regression testing which are based on factors like the types of change introduced and the type of bug which is fixed.

1. Unit Regression Testing: Unit regression testing is executed during the phase of unit testing. As the name suggests, in-unit regression testing the codes are tested like a single unit. This type of testing has a very focused and narrow approach, where the complex dependencies and interactions that are outside the unit of the code which is in question are blocked temporarily.

2. Partial Regression testing: After the process of impact analysis the partial regression testing is performed. In partial regression testing, the coding unit which is newly added is allowed to interact with the parts of the code which is already in use or the pre-existing code unit. This is done to ensure that even after modifying the code the functions of the system are working properly as intended.

3. Complete Regression testing: The complete regression is mostly carried out when there are many changes introduced to the already existing code. It is also used often when the modification in the code or the updates in the software affects the roots or the core of the software due to which a complete or a full system testing is required.

Complete regression testing provides a comprehensive view of your whole system and it will detect and throw out any problem which is left unseen. The final regression testing is performed to ensure that the newly added code has not been changed for a period of time. After testing and fixing the final version is given to all the end-users.

Regression Testing Techniques

Mainly there are 5 types of Regression testing techniques. Each of the techniques is applicable for different purposes. Some technique requires fewer resources while some require a lot of resources but that technique will provide the result with greater details.

The type of technique used in regression testing varies from project to project

and from organization to organization. Some organization prefer to use a particular regression testing technique in all their projects. The 5 main categories of regression testing methods are:

Corrective Regression testing

It is used when no changes are introduced in the current application or software or product specifications. In this case, we can use the test cases which have already been created to perform the testing.

Progressive Regression testing

This technique of regression testing is used in situations when new changes are introduced in the software's specification and new test cases need to be designed to perform the desired tests. The progressive regression testing works very effectively when certain changes are introduced in the software's specification and when new test cases are designed.

In this technique, a part of the test cases is taken from the prevailing test cases and used which reduces the cost and effort involved in designing all the new test cases.

Selective Regression Testing

This technique of regression testing is used to analyze the effects of the newly added code which is introduced in the software to the already existing code in the software. In order to reduce the retesting efforts and cost involved in selective regression testing, we use a part of the current experiments.

In this technique, a test case is re-executed if any new change is introduced in any module or part of the software or application like factors, capacities and so on. In selective regression testing, the condition between the experiment and the elements covered by the program are very difficult to determine.

Retest-all regression testing

As the name suggests this technique of regression testing involves reusing all the test cases and also testing all the possible aspects of the product or software even when a new change is introduced in the software or not.

This technique is time-consuming and very tedious. This is very expensive as it requires a huge amount of resources because we test all the aspects of the software and all the tests in the existing test bucket have to be re-executed.

The retest-all regression testing is not advised to use when only a small change is introduced in the software or the product.

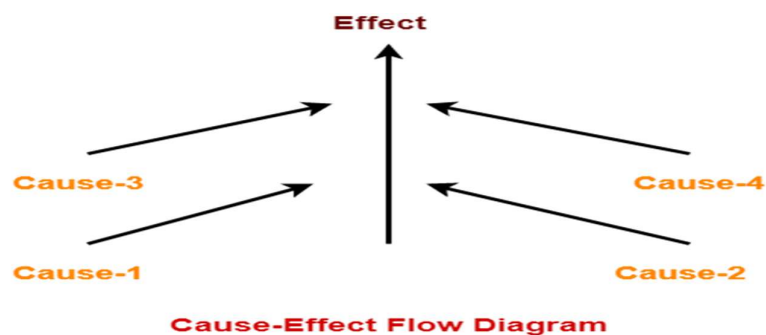
Complete Regression Testing

We have already discussed this in the types of regression testing. The complete regression testing is used more often when multiple changes are introduced in the software and it is also used in the case when the changes made in the software affect the root or the core of the software.

This is beneficial as it helps in identifying the unseen problems. After the testing is complete the final software or the product is provided to the end-users.

Cause Effect Graph

- Cause Effect Graph is a popular black box testing technique.
- It illustrates the relationship between a given outcome and all the factors that influence the outcome graphically.



Here,

- A “Cause” stands for a distinct input condition that fetches about an internal change in the system.
- An “Effect” represents an output condition, a system state that results from a combination of causes.

Applications-

- For analyzing the existing problem so that corrective actions can be taken at the earliest.
- For relating the interactions of the system with the factors affecting a particular process.
- For identifying the possible root causes, reasons for a particular effect, problem or outcome.

Advantages-

- It helps to determine the root causes of a problem or quality.
- It indicates possible causes of variation in a process.
- It identifies those areas where data should be collected for further study.
- It utilizes the team knowledge of the process by encouraging team participation.

Steps For Drawing Cause Effect Diagram-

The following steps are followed-

- Identify and describe the input conditions (causes) and actions (effect).
- Build up a cause-effect graph.
- Convert cause-effect graph into a decision table.
- Convert decision table rules to test cases where each column of the decision table represents a test case.

PRACTICE PROBLEMS BASED ON CAUSE-EFFECT GRAPH TECHNIQUE-

Problem-01:

Design test cases for the following problem-

If the character of the first column is 'A' or 'B' and the second column is a number, then the file is considered updated. If the first character is erroneous, then message x should be printed. If the second column is not a number, then message y should be printed.

Solution-

Step-01:

Identify and describe the input conditions (causes) and actions (effect).

The causes represented by letter "C" are as follows-

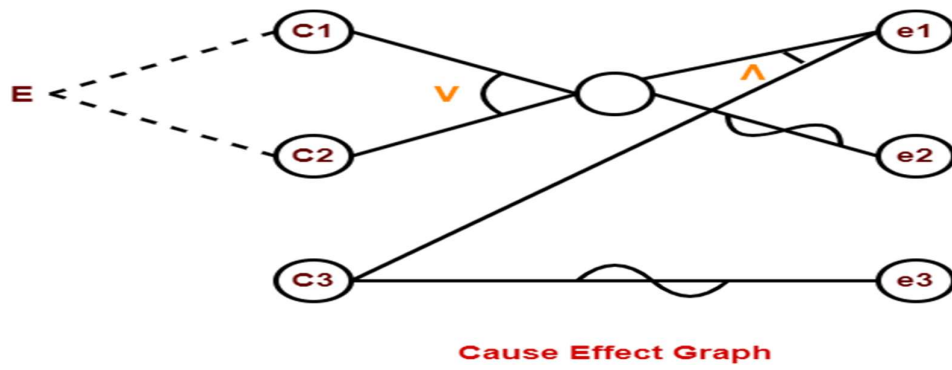
- C1 : The character in column 1 is 'A'
- C2 : The character in column 1 is 'B'
- C3 : The character in column 2 is a number

The effects represented by letter "e" are as follows-

- e1 : File update is made
- e2 : Message x is printed
- e3 : Message y is printed

Step-02:

Build up a cause-effect graph-



Step-03:

Convert cause-effect graph into a decision table-

Test data	Causes			Effect		
	A1	A2	A3	M1	M2	M3
1	0	0	0	0	1	1
2	0	0	1	0	1	0
3	0	1	0	0	0	1
4	0	1	1	1	0	0
5	1	0	0	0	0	1
6	1	0	1	1	0	0

CRUD TESTING

CRUD testing is black-box testing. It is also called as Database Testing. The CRUD operations are:

- Create: to save a new transaction
- Retrieve/Read: to search or view a transaction
- Update: to edit or modify an existing transaction
- Delete/Destroy: to delete an existing unnecessary transaction

CRUD Testing is important for:

- Ensuring Data Mapping
- Maintaining ACID Properties
- Ensuring data Integrity
- Ensuring the accuracy of implemented business rules

Inspections are a formal type of review that involves checking the documents thoroughly before a meeting and is carried out mostly by moderators. A meeting is then held to review the code and the design.

Inspection meetings can be held both physically and virtually. The purpose of these meetings is to review the code and the design with everyone and to report any bugs found.

Benefits

- ☐ It is easier to find defects for the people who have not done the implementation themselves and are unaware of its correctness beforehand.
- ☐ Knowledge sharing about specific software artifacts and designs.
- ☐ Knowledge sharing regarding defect detection practices.
- ☐ Flaws are identified at early stages.
- ☐ It reduces the rework and testing effort.

Who is involved?

- ☐ Moderator: Inspector who is responsible for organizing and reporting on inspection.
- ☐ Author: Owner of the report.
- ☐ Reader: A person who guides the examination of the product.
- ☐ Recorder: An inspector who notes down all the defects on the defect list.
- ☐ Inspector: Member of the inspection team.

Steps of inspection

Planning

The planning phase starts when the entry criteria for the inspection state are met. A moderator verifies that the product entry criteria are met.

Overview

In the overview phase, a presentation is given to the inspector with some background information needed to review the software product properly.

Preparation

This is considered an individual activity. In this part of the process, the inspector collects all the materials needed for inspection, reviews that material, and notes any defects.

Meeting

The moderator conducts the meeting. In the meeting, the defects are collected and reviewed.

Rework

The author performs this part of the process in response to defect disposition determined at the meeting.

Follow-up

In follow-up, the moderator makes the corrections and then compiles the inspection management and defects summary report.

Database Testing

Database Testing is a type of software testing that checks the schema, tables, triggers etc. of the database under test. It involves creating complex queries for performing the load or stress test on the database and check its responsiveness. It checks integrity and consistency of data. Database testing usually consists of a layered process that includes the User Interface (UI) layer, the business layer, the data access layer and the database.

Objective of the Database Testing:

1. Ensure Data Mapping:

It checks whether the fields in the user interface or front end forms are mapped consistently with the corresponding fields in the database table.

2. Ensure ACID Properties of Transactions:

Every transaction a database performs has to stick to these four properties: Atomicity, Consistency, Isolation and Durability.

3. Ensure Data Integrity:

The updated and the most recent values of shared data should appear on all the forms and screens. The value should not be updated on one screen and display an older value on another one. The status should also be updated simultaneously.

4. Ensure the Accuracy of the Business Rules:

Complex databases lead to complicated components like relational constraints, triggers and stored procedures. Hence in order testers come up with appropriate SQL queries to validate the complex objects.

Database Testing Attributes:

1. Transactions:

Transactions mean the access and retrieval of the data. Hence in order during the transaction processes the ACID properties should be followed.

2. Database Schema:

It is the design or the structure about the organization of the data in the database.

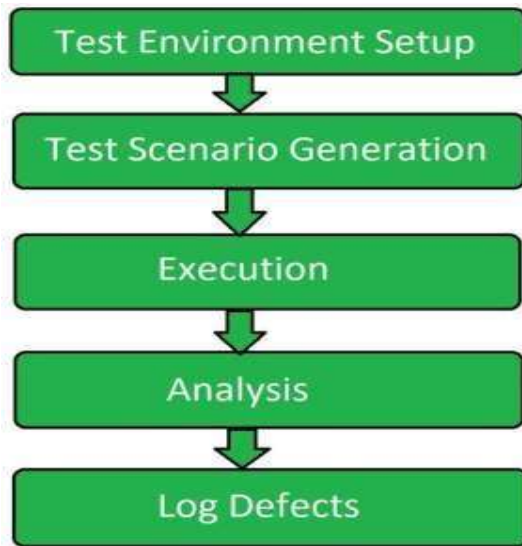
3. Triggers:

When a certain event occurs in a certain table, a trigger is auto-instructed to be executed.

4. Procedures:

It is the collection of the statements or functions governing the transactions in the database.

Database Testing Process:



1. Test Environment Setup:

Database testing starts with the setting up the testing environment for the testing process to be carried out in order to get a good quality testing process.

2. Test Scenario Generation:

After setting up the test environment, test cases are designed for conducting the test. Test scenarios involve the different inputs and different transactions related to the database.

3. Execution:

Execution is the core phase of the testing process in which the testing is conducted. It is basically related to the execution of the test cases designed for the testing process.

4. Analysis:

Once the execution phase is ended, then all the process and the output obtained is analyzed. It is checked whether the testing process has been conducted properly or not.

5. Log Defects:

Log defects are also known as report submitting. In this last phase, the tester informs the developer about the defects found in the database of the system.

MUTATION TESTING

Mutation testing is a technique used in software testing to evaluate the quality of a test suite by intentionally introducing small changes, or *mutations*, into the source code and checking if the tests can detect these changes. The idea is to simulate potential faults in the system to determine how well the existing test cases can catch errors and ensure the robustness of the code.

Here's an overview of how mutation testing works and why it is used:

1. How Mutation Testing Works

- **Create Mutants:** A *mutant* is a software version where a small change is made, typically one that would cause a fault. This change can involve altering operators (e.g., replacing `==` with `!=`), modifying control flow (e.g., changing a while loop to an if statement), or adjusting constants (e.g., changing a value in an array).
- **Run Tests:** The test suite is run against the mutated version (mutant) of the program.
- **Evaluate the Test Results:**
 - If the test suite **fails** (i.e., detects the mutation), the mutant is considered "killed," meaning the test case was able to detect the introduced fault.
 - If the test suite **passes**, the mutant is considered "surviving," meaning the test case did not catch the fault, indicating a potential weakness in the tests.
- **Mutation Score:** The effectiveness of the test suite is measured by the mutation score, which is the percentage of mutants that were killed divided by the total number of mutants. The higher the mutation score, the more effective the test suite is at detecting faults.

2. Types of Mutations

Mutation testing involves various types of mutations, such as:

- **Operator Replacement:** Changing arithmetic, logical, or relational operators. For example, replacing `+` with `-`, or `>` with `<`.
- **Statement Deletion:** Removing a statement or a piece of code entirely.
- **Inserting New Statements:** Introducing unnecessary code or statements that don't affect the program's behavior but might be detected by tests.
- **Variable Replacement:** Changing variable names or values to test the robustness of the tests against logical errors.

3. Benefits of Mutation Testing

- **Improves Test Quality:** Mutation testing helps identify weaknesses in test suites by ensuring that tests are able to detect subtle faults in the code.
- **Enhances Code Coverage:** While traditional code coverage metrics (e.g., statement, branch coverage) measure how much of the code is executed, mutation testing evaluates how well the test suite can detect errors in the code. This gives a deeper insight into the effectiveness of the tests.
- **Better Fault Detection:** It helps in finding undetected faults, especially those that might be missed with traditional testing techniques.
- **Can Lead to More Robust Software:** By identifying test gaps, mutation testing helps ensure the software is more reliable and defect-free.

4. Challenges of Mutation Testing

- **Performance Overhead:** Mutation testing is computationally expensive because it requires generating many mutants and running the test suite against each one. For large systems, this can be time-consuming and resource-intensive.

- **False Positives:** Sometimes, mutations may not truly represent real defects, leading to unnecessary changes in test cases. Mutants that "survive" might still be harmless or unrealistic.
- **High Cost:** The time and resources required to run mutation testing for large codebases can make it a less practical approach in certain contexts, especially when rapid development cycles are involved.

5. Mutation Testing Tools

Several tools are available to automate mutation testing:

- **PIT (for Java):** One of the most popular tools for mutation testing in Java.
- **MutPy (for Python):** A mutation testing tool designed for Python projects.
- **Stryker (for JavaScript, TypeScript, C#, and others):** A mutation testing framework that supports multiple programming languages.
- **Jumble (for Java):** A mutation testing tool for Java programs.

These tools automatically generate mutants, run the tests, and calculate the mutation score.

6. Mutation Score Example

Suppose you have a system with 100 mutants, and your test suite kills 80 of them (detects the fault). The mutation score would be:

Mutation Score = $\frac{80}{100} \times 100 = 80\%$
 $\text{Mutation Score} = \frac{80}{100} \times 100 = 80\%$

This means your test suite successfully detected 80% of the mutations and is considered fairly effective in detecting faults.

Random Testing

Random testing is [software testing](#) in which the system is tested with the help of generating random and independent inputs and test cases. Random testing is also named monkey testing. It is a black box assessment outline technique in which the tests are being chosen randomly and the results are being compared by some software identification to check whether the output is correct or incorrect. Types of Random Testing

1. **Random input sequence generation:** It is also known as Random Number Generator (RNG) in which a random sequential number or symbols is being generated which cannot be assumed during the random selection.
2. **Random sequence of data inputs:** In this, all the data are selected randomly for the inputs which can be used during the testing.
3. **Random data selection from an existing database:** The record where all the data are available from that record only the data can be selected for testing afterward no additional data cannot be added which are not available in the record.

Characteristics of Random Testing:

1. Random testing is implemented when the bug in an application is not recognized.
2. It is used to check the system's execution and dependability.
3. It saves our time and does not need any extra effort.
4. Random testing is less costly, it doesn't need extra knowledge for

testing the program.

Methods to Implement Random Testing:

To implement the random testing basically, four steps are applied:

1. The user input domain is analyzed.
2. After that, from that domain, the data of test inputs are chosen separately.
3. With the help of these test inputs, the test is executed successfully. These input tests conduct random sets of tests.
4. The outcomes are compared with the system identification. The outcome of the test becomes unsuccessful if any test input doesn't match with the original one otherwise the outcomes are always successful.

Advantages of Random Testing

1. It is very cheap so that anyone can use this software.
2. It doesn't need any special intelligence to access the program during the tests.
3. Errors can be traced very easily; it can easily detect the bug throughout the testing.

Disadvantages of Random Testing

1. This software only finds changes errors.
2. They are not practical. Some tests will be of no use for a longer time.
3. Most of the time is consumed by analyzing all the tests.
4. New tests cannot be formed if their data is not available during testing.

Tools used for Random Testing

1. QuickCheck: It is a famous test tool, which was introduced for Haskell which is available in many different languages. This tool generates random orders for API calls that are related to the mode and the properties of the system which can give successful results after every test.
2. Randoop: This tool provides an order of methods and constructor acknowledgment for the classes during the test and generates JUnit tests.
3. Simulant: It is a Clojure tool that runs according to the system's different specifications, the behavior of the model.

performance Testing

Performance Testing is a software testing process used for testing the speed, response time, stability, reliability, scalability and resource usage of a software application under particular workload. The main purpose of performance testing is to identify and eliminate the performance bottlenecks in the software application. It is a subset of performance engineering and also known as “Perf Testing”.

The focus of Performance Testing is checking a software program’s

- Speed – Determines whether the application responds quickly
- Scalability – Determines maximum user load the software application can handle.
- Stability – Determines if the application is stable under varying loads

Types of Performance Testing

- Load testing – checks the application’s ability to perform under anticipated user loads. The objective is to identify performance bottlenecks before the software application goes live.
- Stress testing – involves testing an application under extreme workloads to see how it handles high traffic or data processing. The objective is to identify the breaking point of an application.
- Endurance testing – is done to make sure the software can handle the expected load over a long period of time.
- Spike testing – tests the software’s reaction to sudden large spikes in the load generated by users.
- Volume testing – Under Volume Testing large no. of. Data is populated in a database and the overall software system’s behavior is monitored. The objective is to check software application’s performance under varying database volumes.
- Scalability testing – The objective of scalability testing is to determine the software application’s effectiveness in “scaling up” to support an increase in user load. It helps plan capacity addition to your software system.

Performance Testing Process

The methodology adopted for performance testing can vary widely but the objective for performance tests remain the same. It can help demonstrate that your software system meets certain pre-defined performance criteria. Or it can help compare the performance of two software systems. It can also help identify parts of your software system which degrade its performance.

Below is a generic process on how to perform performance testing



1. Identify your testing environment – Know your physical test environment, production environment and what testing tools are available. Understand details of the hardware, software and network configurations used during testing before you begin the testing process. It will help testers create more efficient tests. It will also help identify possible challenges that testers may encounter during the performance testing procedures.
2. Identify the performance acceptance criteria – This includes goals and constraints for throughput, response times and resource allocation. It is also necessary to identify project success criteria outside of these goals and constraints. Testers should be empowered to set performance criteria and goals because often the project specifications will not include a wide enough variety of performance benchmarks. Sometimes there may be none at all. When possible finding a similar application to compare to is a good way to set performance goals.
3. Plan & design performance tests – Determine how usage is likely to vary amongst end users and identify key scenarios to test for all possible use cases. It is necessary to simulate a variety of end users, plan performance test data and outline what metrics will be gathered.
4. Configuring the test environment – Prepare the testing environment before execution. Also, arrange tools and other resources.
5. Implement test design – Create the performance tests according to your test design.
6. Run the tests – Execute and monitor the tests.
7. Analyze, tune and retest – Consolidate, analyze and share test results. Then fine tune and test again to see if there is an improvement or decrease in performance. Since improvements generally grow smaller with each retest, stop when bottlenecks are caused by the CPU. Then you may have the consider option of increasing CPU power.

stress Testing

Stress Testing is a type of software testing that verifies stability & reliability of software application. The goal of Stress testing is measuring software on its robustness and error handling capabilities under extremely heavy load conditions and ensuring that software doesn't crash under crunch situations. It even tests beyond normal operating points and evaluates how software works under extreme conditions.

Types of Stress Testing:

Following are the types of stress testing and are explained as follows:

Distributed Stress Testing:

In distributed client-server systems, testing is done across all clients from the server. The role of stress server is to distribute a set of stress tests to all stress clients and track on the status of the client. After the client contacts the server, the server adds the name of the client and starts sending data for testing.

Meanwhile, client machines send signal or heartbeat that it is connected with the server. If the server does not receive any signals from the client machine, it needs to be investigated further for debugging. From the figure, a server can connect with the 2 clients (Client1 and Client2), but it cannot send or receive a signal from Client 3 & 4.

Night run is the best option to run these stress testing scenarios. Large server farms need a more efficient method for determining which computers have had stress failures that need to be investigated.

Application Stress Testing:

This testing concentrate on finding defects related to data locking and blocking, network issues and performance bottlenecks in an application.

Transactional Stress Testing:

It does stress testing on one or more transactions between two or more applications. It is used for fine-tuning & optimizing the system.

Systemic Stress Testing:

This is integrated stress testing which can be tested across multiple systems running on the same server. It is used to find defects where one application data blocks another application.

Exploratory Stress Testing:

This is one of the types of stress testing which is used to test the system with

unusual parameters or conditions that are unlikely to occur in a real scenario. It is used to find defects in unexpected scenarios like

1. A large number of users logged at the same time
2. If a virus scanner started in all machines simultaneously
3. If Database has gone offline when it is accessed from a website,
4. When a large volume of data is inserted to the database simultaneously

How to do Stress Testing?

Stress Testing process can be done in 5 major steps:

1. Planning the Stress Test. Here you gather the system data, analyze the system, define the stress test goals
2. Create Automation Scripts: In this phase, you create the Stress testing automation scripts, generate the test data for the stress scenarios.
3. Script Execution: In this stage, you run the Stress testing automation scripts and store the stress results.
4. Results Analysis: In this stage, you analyze the Stress Test results and identify bottlenecks.
5. Tweaking and Optimization: In this stage, you fine-tune the system, change configurations, optimize the code with goal meet the desired benchmark.

Tools:

[LoadRunner](#)

LoadRunner from HP is a widely-used Load Testing tool. Load Test Results shaped by Loadrunner are considered as a benchmark.

[Jmeter](#)

Jmeter is an Open Source testing tool. It is a pure [Java](#) application for stress and [Performance Testing](#). Jmeter is intended to cover types of tests like load, functional, stress, etc. It needs JDK 5 or higher to function.

Stress Tester

This tool provides extensive analysis of the web application performance, provides results in graphical format, and it is extremely easy to use. No high-level scripting is required and gives a good return on investment.

Neo load

This is a popular tool available in the market to test the web and [Mobile](#) applications. This tool can simulate thousands of users in order to

evaluate the application performance under load and analyze the response times. It also supports Cloud-integrated – performance, load and stress testing. It is easy to use, cost-effective and provides good scalability.

SPIKE TESTING

Spike Testing is a type of performance testing used to evaluate the behavior of a system under extreme or sudden increases in load or traffic. The primary goal of spike testing is to determine how the system responds to unexpected or drastic changes in load and to identify any potential performance bottlenecks or failures that could occur under such conditions.

Key Characteristics of Spike Testing:

- **Sudden Load Changes:** Unlike stress testing (which gradually increases the load until the system fails), spike testing involves quickly and unexpectedly increasing the system load in a short period, often followed by a rapid decrease.
- **Focus on Performance and Stability:** The aim is to observe how the system reacts when subjected to sudden spikes in traffic, such as server crashes, slow responses, or failures in critical components.
- **Short Duration:** Spike testing is generally conducted over a short period, often to simulate short bursts of high traffic or load.

When to Use Spike Testing:

- **Sudden Surge in Traffic:** It is typically used to test how a system performs under sudden surges in traffic or load, like during product launches, flash sales, or unexpected viral events.
- **High Risk Scenarios:** Systems that require high availability and stability, such as e-commerce platforms, banking applications, or social media sites, benefit from spike testing to prepare for sudden increases in user interactions.
- **Capacity Planning:** It helps identify the upper limits of the system's capacity and how it can scale in real-time under pressure.

How Spike Testing Works:

1. **Define Baseline:** Start by establishing a baseline performance of the system under normal operating conditions, with no added load or stress.
2. **Apply Sudden Load:** Introduce a sharp, sudden increase in the number of requests, transactions, or users interacting with the system. This can simulate real-world scenarios like viral traffic spikes.
3. **Monitor System Behavior:** Observe how the system behaves under the increased load. Key metrics to monitor include response times, resource utilization (e.g., CPU, memory), throughput, error rates, and availability.
4. **Decrease Load Rapidly:** After the load spike, rapidly reduce the load back to normal levels and observe how the system recovers, including whether it returns to normal performance levels.
5. **Analyze Results:** Evaluate the system's stability and performance, and identify any issues, such as crashes, slowdowns, or resource exhaustion. Focus on whether the system can handle and recover from sudden spikes without significant degradation.

Example Scenario of Spike Testing:

- **E-Commerce Website During Flash Sale:** Suppose an e-commerce platform prepares for a flash sale with a limited number of items at a heavily discounted price. A sudden influx of thousands of users trying to access the website at the same time could cause performance issues, such as slower page loads, server crashes, or even application failures.
- Spike testing would simulate this situation by quickly ramping up the number of simultaneous users and measuring how well the system handles the load, both during the spike and in its recovery.

Benefits of Spike Testing:

1. **Performance Validation:** Helps verify whether the system can handle sudden, unexpected increases in traffic and whether it will continue to function as expected.
2. **Scalability Insights:** Provides insights into how the system scales under sudden load, which is especially valuable for cloud-based or distributed systems that need to manage fluctuating demands.
3. **Risk Mitigation:** Helps uncover potential failure points in critical system components, allowing developers to address issues before they occur in a real-world scenario.
4. **Recovery Testing:** Evaluates how well the system recovers from extreme loads, which is crucial for ensuring uptime and availability during traffic surges.

Challenges of Spike Testing:

- **Unpredictability:** Simulating real-world traffic spikes can be challenging because such events are unpredictable. Therefore, it may not always be possible to exactly replicate the load conditions experienced during actual traffic spikes.
- **Resource Intensive:** Testing with high volumes of traffic can strain the infrastructure and resources, especially when attempting to simulate extreme conditions.
- **Time Constraints:** Because spike testing is conducted over short durations, it may not always capture the full extent of system performance under prolonged load conditions.

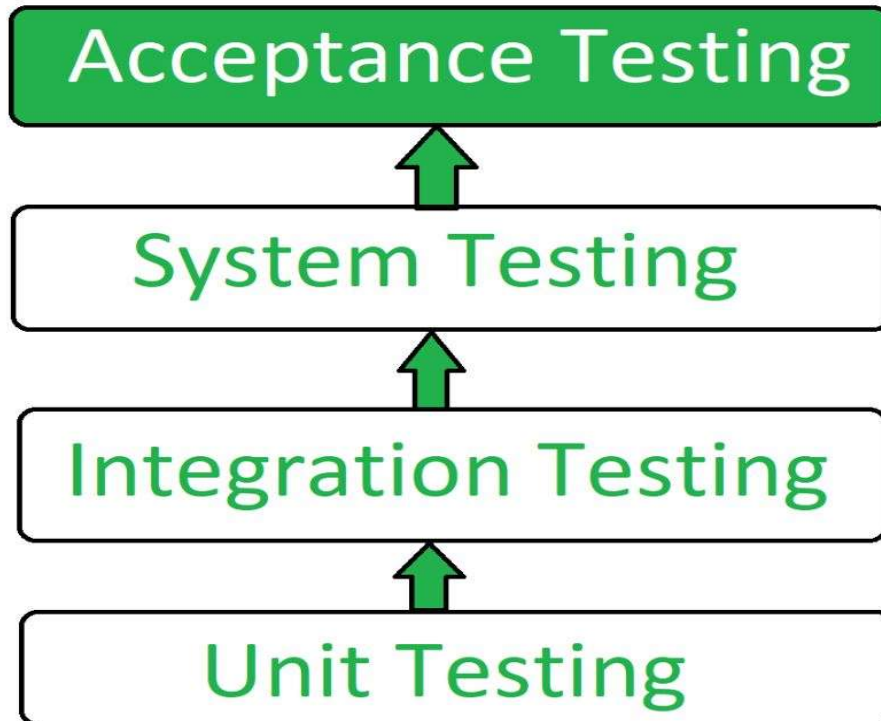
Example Tools for Spike Testing:

1. **Apache JMeter:** A popular open-source tool for performance testing that can simulate sudden spikes in traffic and measure the system's response.
2. **LoadRunner:** A performance testing tool by Micro Focus that can simulate user traffic spikes and analyze system behavior under stress.
3. **Gatling:** Another tool for load testing that allows users to simulate sudden traffic spikes and test server performance.
4. **BlazeMeter:** A cloud-based load testing tool that supports spike testing and performance monitoring.

Acceptance Testing

It is a formal testing according to user needs, requirements and business processes conducted to determine whether a system satisfies the acceptance criteria or not and to enable the users, customers or other authorized entities to determine whether to accept the system or not.

Acceptance Testing is the last phase of software testing performed after System Testing and before making the system available for actual use.



Types of Acceptance Testing:

1. **User Acceptance Testing (UAT):** User acceptance testing is used to determine whether the product is working for the user correctly. Specific requirements which are quite often used by the customers are primarily picked for the testing purpose. This is also termed as End-User Testing.
2. **Business Acceptance Testing (BAT):** BAT is used to determine whether the product meets the business goals and purposes or not. BAT mainly focuses on business profits which are quite challenging due to the changing market conditions and new technologies so the current implementation may have to be changed which results in extra budgets.
3. **Contract Acceptance Testing (CAT):** CAT is a contract that specifies that once the product goes live, within a predetermined period, the acceptance test must be performed and it should pass all the acceptance use cases. Here is a contract termed a Service Level Agreement (SLA), which includes the terms where the payment will be made only if the Product services are in-line with all the requirements, which means the contract is fulfilled. Sometimes, this contract happens before the product goes live. There should be a well-defined contract in terms of the period of testing, areas of testing, conditions on issues encountered at later stages, payments, etc.
4. **Regulations Acceptance Testing (RAT):** RAT is used to determine whether the product violates the rules and regulations that are defined by the government of the country

where it is being released. This may be unintentional but will impact negatively on the business. Generally, the product or application that is to be released in the market, has to go under RAT, as different countries or regions have different rules and regulations defined by its governing bodies. If any rules and regulations are violated for any country then that country or the specific region then the product will not be released in that country or region. If the product is released even though there is a violation then only the vendors of the product will be directly responsible.

5. Operational Acceptance Testing (OAT): OAT is used to determine the operational readiness of the product and is non-functional testing. It mainly includes testing of recovery, compatibility, maintainability, reliability, etc. OAT assures the stability of the product before it is released to production.
6. Alpha Testing: Alpha testing is used to determine the product in the development testing environment by a specialized testers team usually called alpha testers.
7. Beta Testing: Beta testing is used to assess the product by exposing it to the real end-users, usually called beta testers in their environment. Feedback is collected from the users and the defects are fixed. Also, this helps in enhancing the product to give a rich user experience.

Use of Acceptance Testing:

- To find the defects missed during the functional testing phase.
- How well the product is developed.
- A product is what actually the customers need.
- Feedback help in improving the product performance and user experience.
- Minimize or eliminate the issues arising from the production.

Advantages of Acceptance Testing :

- This testing helps the project team to know the further requirements from the users directly as it involves the users for testing.
- Automated test execution.
- It brings confidence and satisfaction to the clients as they are directly involved in the testing process.
- It is easier for the user to describe their requirement.
- It covers only the Black-Box testing process and hence the entire functionality of the product will be tested.

Disadvantages of Acceptance Testing :

- Users should have basic knowledge about the product or application.
- Sometimes, users don't want to participate in the testing process.
- The feedback for the testing takes long time as it involves many users and the opinions may differ from one user to another user.
- Development team is not participated in this testing process.

Alpha Testing and Beta Testing

Alpha Testing and **Beta Testing** are two important stages of software testing that occur towards the end of the software development life cycle. Both are types of **user acceptance testing** (UAT), but they differ in terms of who conducts the testing, when they occur, and the scope of feedback they aim to gather.

Alpha Testing

Alpha testing is the **first phase** of testing performed by internal testers (developers or quality assurance engineers) before the software is released to a select group of external users. It is usually carried out in-house at the development company.

Key Characteristics:

- **Who Conducts It:** Internal teams, typically developers and quality assurance (QA) testers, perform alpha testing.
- **When It Happens:** Alpha testing occurs after the software has passed the unit and integration testing phases, and before it is released to external users for beta testing.
- **Environment:** It is performed in a controlled, internal environment, often in a lab or testing setup.
- **Focus:** The focus is on finding bugs and issues that were not identified during earlier testing phases (e.g., unit or integration testing). Alpha testing also checks the overall functionality of the software and its compliance with specified requirements.
- **Test Types:** It often involves both **black-box** and **white-box** testing, including functionality testing, interface testing, and performance testing.

Goals of Alpha Testing:

- **Bug Detection:** The primary goal is to identify bugs and defects in the software before it is released to real users.
- **Usability Feedback:** Gather feedback on the usability of the software, including user interface design, and ease of use.
- **Verification:** Verify whether the software meets the specified requirements and functions as expected in a controlled environment.
- **Security:** Test for security vulnerabilities or issues in the application.

Process:

1. **Test Planning:** QA teams create test plans based on the specifications and expected functionality of the software.
2. **Execution:** Test cases are executed based on the test plan. Developers may also perform exploratory testing.
3. **Bug Reporting:** All issues discovered during testing are documented, and feedback is provided to the development team for resolution.
4. **Fixing Issues:** Developers fix the bugs identified during alpha testing and prepare for beta testing.

Beta Testing

Beta testing is the **second phase** of testing that involves releasing the software to a limited number of external users, known as "beta testers." These testers are typically end users who have not been involved in the software's development process.

Key Characteristics:

- **Who Conducts It:** Beta testing is conducted by external users (actual or potential customers, end-users, or other stakeholders).

- **When It Happens:** Beta testing occurs after alpha testing and when the software is considered feature-complete but may still have some issues. It typically happens just before the final release of the software.
- **Environment:** It is performed in a real-world environment, with users testing the software on their own hardware and systems.
- **Focus:** The goal is to get feedback on the user experience, identify any remaining bugs, assess performance under real-world conditions, and gather feedback on the software's usability.

Goals of Beta Testing:

- **Real-World Feedback:** To obtain feedback from end-users regarding usability, functionality, and performance in real-world usage.
- **Bug Identification:** To find bugs that were not discovered during alpha testing and to validate if the software is stable enough for public release.
- **Performance Validation:** To see how the software performs under actual user conditions and environments (e.g., different hardware, network speeds, etc.).
- **Usability Testing:** To check whether the software is user-friendly and if there are any design improvements that could enhance the user experience.
- **Market Readiness:** To evaluate the software's readiness for release, both in terms of features and stability.

Process:

1. **Release to Beta Testers:** The software is released to a select group of external users, often through a closed beta (invite-only) or open beta (available to anyone).
2. **Test Execution:** Beta testers use the software in their own environments, typically focusing on specific features or functions.
3. **Feedback Collection:** Testers provide feedback through bug reports, surveys, or forums, detailing issues encountered, suggestions for improvements, and overall experiences.
4. **Fixing Issues:** Based on feedback, the development team resolves any issues identified, makes necessary improvements, and prepares the software for final release.
5. **Release Candidate:** After beta testing, the software is refined into a "release candidate" and is considered ready for final deployment to the public.

Differences Between Alpha and Beta Testing

Aspect	Alpha Testing	Beta Testing
Testers	Internal testers (QA team, developers)	External testers (end-users, customers)
Timing	Done before the product is released to the public	Done after alpha testing, before final release
Environment	Controlled internal environment	Real-world environment, external devices and systems
Goal	Find bugs, verify functionality, and ensure software meets requirements	Gather real-world feedback, identify any final bugs, and ensure usability
Focus	Detecting critical bugs and compliance with specifications	Usability, performance, and user experience testing

Aspect	Alpha Testing	Beta Testing
Tools	Internal testing tools, bug tracking systems	Feedback forms, surveys, bug tracking systems
Feedback Type	Internal technical feedback from the team	External feedback from actual users

Benefits of Alpha and Beta Testing:

1. Alpha Testing:

- Helps identify major bugs early before the software is exposed to real users.
- Ensures that the software meets the basic functional and technical requirements.
- Validates the system's core features and overall design.

2. Beta Testing:

- Provides real-world insights into how the software will perform with actual users.
- Identifies any bugs or issues that were missed during earlier stages.
- Helps improve the user experience and ensures the software is ready for release.
- Builds confidence in the software by getting early feedback from potential customers.

Automated Testing Tools: Need for Automated Testing Tools, Taxonomy of Testing Tools, Exposure to Software Testing Tools: Selenium, UFT.

Automated Testing Tools: Overview

Automated testing tools are software programs or frameworks designed to automatically perform pre-scripted tests on a software application. They help accelerate the testing process, improve efficiency, and enhance the reliability of software applications. With the growing complexity of modern software systems, manual testing is becoming increasingly time-consuming and prone to human error, making automation a critical necessity for many organizations.

Need for Automated Testing Tools

- 1. Increased Efficiency and Speed:** Automated testing can execute test cases much faster than manual testing. A large number of test cases can be run in a short period, allowing for frequent regression testing without significant time investment.

2. **Reusability of Test Scripts:** Once automated test scripts are created, they can be reused across different builds, environments, or software versions. This reusability reduces the time required for testing new software iterations.
3. **Consistency and Accuracy:** Automated tests can be executed consistently without variation, ensuring that the same steps are followed every time. This eliminates human errors and biases associated with manual testing.
4. **Cost-Effective in the Long Run:** Although initial setup and scripting can be resource-intensive, over time automated tests reduce the need for manual effort and can save costs in terms of time and resources, particularly for large and complex systems.
5. **Supports Continuous Integration (CI) / Continuous Delivery (CD):** Automated testing tools integrate seamlessly with CI/CD pipelines, enabling continuous testing throughout the development lifecycle. This ensures quicker delivery of reliable software.
6. **Handling Repetitive Tasks:** Automation is especially useful for repetitive tests such as smoke testing, regression testing, and performance testing, which can be time-consuming if done manually.
7. **Improved Test Coverage:** Automated tests can execute more test cases and scenarios than manual testing, providing better test coverage and uncovering more bugs.

Taxonomy of Testing Tools

Automated testing tools can be classified based on various factors like the type of testing they support, the technology they are based on, or their usage within the software development lifecycle. Here's a common classification:

1. Functional Testing Tools

- **Purpose:** Validate the application's functionality against defined requirements.
- **Examples:** Selenium, QTP/UFT, TestComplete.

2. Performance Testing Tools

- **Purpose:** Assess how the system performs under load, stress, or heavy traffic conditions.
- **Examples:** JMeter, LoadRunner, Gatling.

3. Regression Testing Tools

- **Purpose:** Ensure that new code changes do not introduce defects into existing functionality.
- **Examples:** Selenium, Rational Functional Tester, UFT.

4. Unit Testing Tools

- **Purpose:** Test individual units or components of the application to ensure they work as expected.
- **Examples:** JUnit, NUnit, TestNG.

5. Security Testing Tools

- **Purpose:** Identify vulnerabilities and security flaws in the software.
- **Examples:** OWASP ZAP, Burp Suite.

6. API Testing Tools

- **Purpose:** Test the interaction between software components via their APIs.
- **Examples:** Postman, SoapUI, Rest Assured.

7. Mobile Testing Tools

- **Purpose:** Automated testing specifically designed for mobile applications.
- **Examples:** Appium, Espresso, Calabash.

8. GUI Testing Tools

- **Purpose:** Automated testing for the graphical user interface (GUI) of an application.
- **Examples:** Selenium WebDriver, UFT.

9. Continuous Integration (CI) Tools

- **Purpose:** Integrate automated tests into the CI/CD pipeline for continuous testing.
- **Examples:** Jenkins, Bamboo, Travis CI.

Exposure to Software Testing Tools: Selenium & UFT

Selenium

- **Overview:** Selenium is an open-source framework used for automating web applications across different browsers and platforms. It supports multiple programming languages like Java, Python, Ruby, C#, and JavaScript. Selenium WebDriver, the most popular component, provides a simple interface to interact with web elements for test automation.
- **Key Features:**
 - **Cross-Browser Testing:** Supports all major browsers, including Chrome, Firefox, Safari, and Internet Explorer.
 - **Multi-Language Support:** Supports various programming languages like Java, Python, C#, and JavaScript, making it flexible for developers and testers with different skill sets.
 - **Integration with Tools:** Can be integrated with various CI tools like Jenkins, build tools like Maven, and testing frameworks like TestNG and JUnit.
 - **Cross-Platform Testing:** Selenium can be used on different operating systems like Windows, macOS, and Linux.
 - **WebDriver API:** Provides a set of commands for simulating user actions on the web browser.
- **Use Cases:**
 - Functional testing of web applications.
 - Regression testing.
 - Cross-browser testing for different platforms.

Unified Functional Testing (UFT)

- **Overview:** UFT, formerly known as QTP (QuickTest Professional), is a commercial automated testing tool developed by Micro Focus. It provides functional and

regression testing for web, mobile, and desktop applications. UFT offers a record-and-playback feature that allows testers to create scripts quickly, along with the ability to write more complex test scripts in VBScript.

- **Key Features:**

- **Record and Playback:** Easily record user interactions with the application and play them back for test execution.
- **Cross-Technology Support:** Supports various technologies such as web, desktop, SAP, Oracle, and more.
- **Integrated with ALM:** UFT integrates with Micro Focus ALM (Application Lifecycle Management) for test management and reporting.
- **Object Repository:** Uses a shared object repository for storing and managing application objects, improving reusability and maintainability.
- **Keyword-Driven Testing:** Provides the ability to create tests without writing code, using keywords and actions.

- **Use Cases:**

- Functional and regression testing of web, desktop, and mobile applications.
- Automating business process testing in enterprise applications.
- Testing complex applications requiring deep integration.